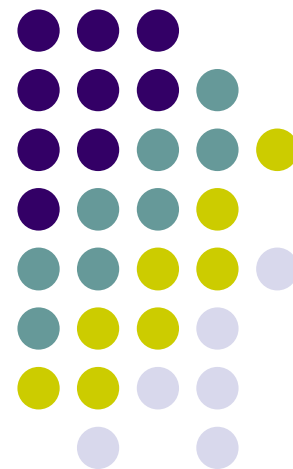
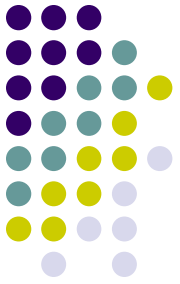


第3章 动态规划

Second Part





最大子段和问题

最大子段和问题



问题描述:

给定由n个整数组成的序列 $a_1, a_2, \dots, a_n$ ，求该序列子段和的最大值。

当所有整数均为负值时定义其最大子段和为0。

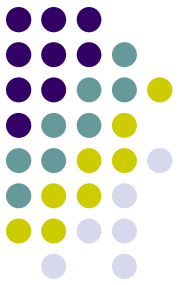
依此定义，所求的最优值为：

$$\max \left\{ 0, \max_{1 \leq i \leq j \leq n} \sum_{k=i}^j a_k \right\}$$

例如，当 $(a_1, a_2, a_3, a_4, a_5, a_6) = (-2, 11, -4, 13, -5, -2)$ 时，最大子段和为：

$$\sum_{k=2}^4 a_k = 11 + (-4) + 13 = 20$$

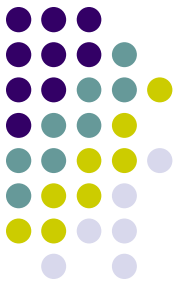
最大子段和问题



一个简单算法:

```
int MaxSum(int n, int a[ ], int &besti, int &bestj) {  
    int i, j, k, sum=0;  
    for (i=0; i<n; i++)  
        for (j=0; j<n; j++) {  
            int thissum=0;  
            for (k=i; k<=j; k++) thissum+=a[k];  
            if (thissum>sum) {  
                sum=thissum;  
                besti=i+1;  
                bestj=j+1;  
            }  
        }  
    return sum;  
} // 时间复杂度为 $O(n^3)$ 
```

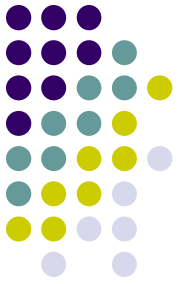
最大子段和问题



简单算法的改进:

```
int MaxSum(int n, int a[], int &besti, int &bestj) {  
    int i, j, sum=0;  
    for (i=0; i<n; i++) {  
        int thissum=0;  
        for (j=i; j<n; j++) {  
            thissum+=a[j];  
            if (thissum>sum) {  
                sum=thissum;  
                besti=i+1;  
                bestj=j+1;  
            }  
        }  
    }  
    return sum;  
} // 时间复杂度为 $O(n^2)$ 
```

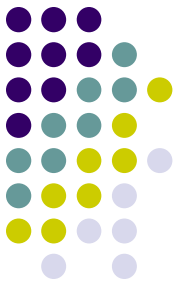
最大子段和问题



分治算法:

将最大子段和 $a[1:n]$ 分为 $a[1:n/2]$ 和 $a[n/2+1 : n]$ 两段,

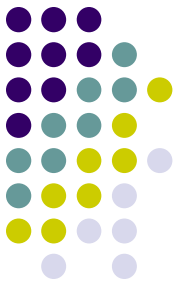
最大子段和问题



分治算法:

```
int MaxSubSum(int a[ ], int left, int right) {  
    int sum=0;  
    if (left==right) sum=a[left]>0 ? a[left] : 0;  
    else {  
        int center=(left+right)/2;  
        int leftsum=MaxSubSum(a,left,center);  
        int rightsum=MaxSubSum(a,center+1,right);  
        int s1=0,lefts=0;  
        for (int i=center; i>=left; i--) {  
            lefts+=a[i];  
            if (lefts>s1) s1=lefts;  
        }  
    }  
}
```

最大子段和问题



分治算法(续):

```
int s2=0,rights=0;
for (i=center+1; i<=right; i++) {
    rights+=a[i];
    if (rights>s2) s2=rights;
}
sum=s1+s2;
if (sum<leftsum) sum=leftsum;
if (sum<rightsum) sum=rightsum;
}
return sum;
}
```

复杂性分析:

$$T(n) = \begin{cases} O(1) & n \leq c \\ 2T\left(\frac{n}{2}\right) + O(n) & n > c \end{cases}$$
$$T(n) = O(n \log n)$$

最大子段和问题



动态规划算法:

在上述分治算法中,

若记 $b_j = \max_{1 \leq i \leq j} \left\{ \sum_{k=i}^j a_k, 1 \leq j \leq n \right\}$, 则所求的最大子段和 为

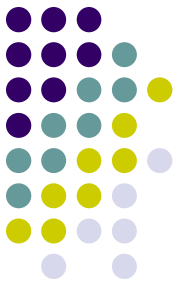
$$\max_{1 \leq i \leq j \leq n} \sum_{k=i}^j a_k = \max_{1 \leq j \leq n} \max_{1 \leq i \leq j} \sum_{k=i}^j a_k = \max_{1 \leq j \leq n} b_j$$

由 b_j 的定义易知, 当 $b_{j-1} > 0$ 时, $b_j = b_{j-1} + a_j$, 否则, $b_j = a_j$ 。

由此可得, 计算 b_j 的动态规划递归式 $b_j = \max\{b_{j-1} + a_j, a_j\}$, $1 \leq j \leq n$ 。

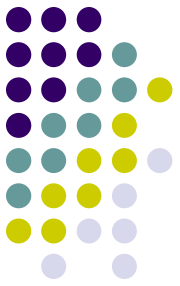
据此, 可设计出求最大子段和的动态规划算法如下:

最大子段和问题



动态规划算法:

```
int MaxSum(int n, int a[ ]) {  
    int i, sum=0, b=0;  
    for (i=0; i<n; i++) {  
        if (b>0) b+=a[i];  
        else b=a[i];  
        if (b>sum) sum=b;  
    }  
    return sum;  
} // 时间复杂度为O(n)
```



最长公共子序列问题

最长公共子序列问题



问题描述:

给定2个序列 $X = \{x_1, x_2, \dots, x_m\}$ 和 $Y = \{y_1, y_2, \dots, y_n\}$, 找出 X 和 Y 的最大长度公共子序列。

Example:

In biological application, given two DNA sequences, for instance

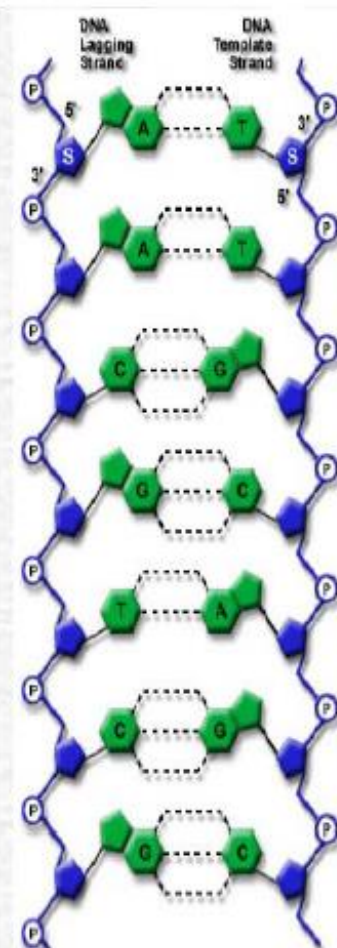
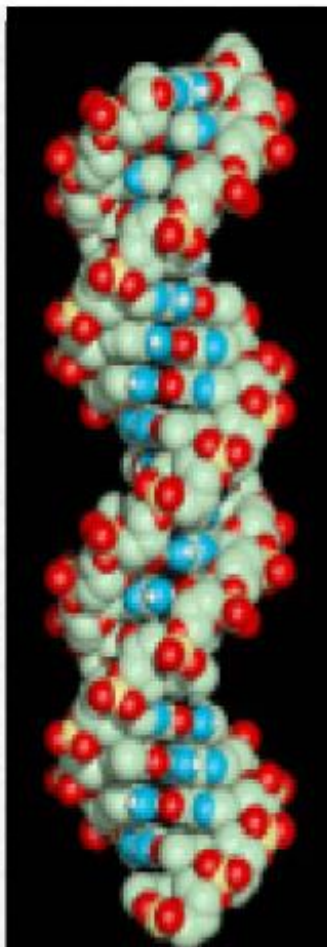
$S_1 =$
ACCGGTCGAGTGCGCGGAAGCCG
GCCGAA

and

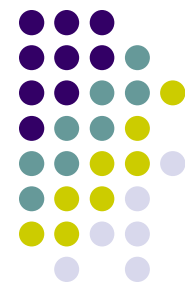
$S_2 =$
GTCGTTCGGAATGCCGTTGCTCT
GTAAA,

how to compare them?

We have various standards of similarity for distinct purposes. While, the LCS of S_1 and S_2 is $S_3 =$ GTCGTCGGAAGCCGGCCGAA.



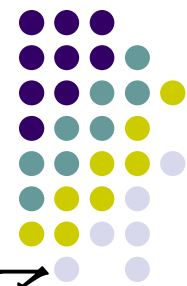
最长公共子序列问题



- 对于给定序列 $X=\{x_1, x_2, \dots, x_m\}$ ，如果另一序列 $Z=\{z_1, z_2, \dots, z_k\}$ 是 X 的子序列，是指存在一个严格递增下标序列 $\{i_1, i_2, \dots, i_k\}$ ，使得对于所有 $j=1, 2, \dots, k$ ，有 $z_j=x_{i_j}$ 。

例如，序列 $Z=\{B, C, D, B\}$ 是序列 $X=\{A, B, C, B, D, A, B\}$ 的子序列，相应的递增下标序列为 $\{2, 3, 5, 7\}$ 。

最长公共子序列问题

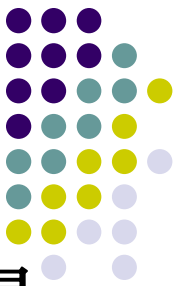


- 给定2个序列X和Y，当另一个序列Z既是X的子序列又是Y的子序列时，称Z是序列X和Y的**公共子序列**。

- **问题描述：**

给定2个序列 $X=\{x_1, x_2, \dots, x_m\}$ 和 $Y=\{y_1, y_2, \dots, y_n\}$ ，
找出X和Y的最长公共子序列。

1. 最长公共子序列的结构



设序列 $X=\{x_1, x_2, \dots, x_m\}$ 和 $Y=\{y_1, y_2, \dots, y_n\}$ 的最长公共子序列为 $Z=\{z_1, z_2, \dots, z_k\}$ ，则

(1) 若 $x_m=y_n$ ，则 $z_k=x_m=y_n$ ，且 z_{k-1} 是 x_{m-1} 和 y_{n-1} 的最长公共子序列。

(2) 若 $x_m \neq y_n$ 且 $x_m < y_n$ ，则 z_k 是 x_m 和 y_{n-1} 的最长公共子序列。由此可见，2个序列的最长公共子序列包含了这2个序列的前缀的最长公共子序列。因此，最长公共子序列问题具有最优子结构性质。

(3) 若 $x_m \neq y_n$ 且 $x_m > y_n$ ，则 z_k 是 x_{m-1} 和 y_n 的最长公共子序列。



Th15.1 的证明

(1) 若 $x_m = y_n$, $\implies z_k = x_m = y_n$ 且 Z_{k-1} 是 X_{m-1} 和 Y_{n-1} 的一个 LCS;
(应用反证法)

先证: $z_k = x_m = y_n$ 。

若 $z_k \neq x_m$ (也有 $z_k \neq y_n$), 则将 x_m 加到 Z 后, 于是获得 X 和 Y 的长度为 $k+1$ 的 CS, 与 Z 是 X 和 Y 的 LCS 矛盾。

$\implies z_k = x_m = y_n$

再证: Z_{k-1} 是 X_{m-1} 和 Y_{n-1} 的一个 LCS。

由 Z 的定义 $\implies$ 前缀 Z_{k-1} 是 X_{m-1} 和 Y_{n-1} 的 CS (长度为 $k-1$)

若 Z_{k-1} 不是 X_{m-1} 和 Y_{n-1} 的 LCS, 则存在一个 X_{m-1} 和 Y_{n-1} 的公共子序列 W , W 的长度 $> k-1$, 于是将 z_k 加入 W 之后, 则产生的公共子序列长度 $> k$, 与 Z 是 X 和 Y 的 LCS 矛盾。

$\implies Z_{k-1}$ 是 X_{m-1} 和 Y_{n-1} 的一个 LCS



● Th15.1 的证明

(2) 若 $x_m \neq y_n$ 且 $z_k \neq x_m$, $\Rightarrow Z$ 是 X_{m-1} 和 Y 的一个 LCS;

$\because z_k \neq x_m$, 则 Z 是 X_{m-1} 和 Y 的一个 CS

下证: Z 是 X_{m-1} 和 Y 的 LCS

(反证) 若不然, 则存在长度 $> k$ 的 CS 序列 W ,
显然, W 也是 X 和 Y 的 CS, 但其长度 $> k$, 矛盾。

(3) 若 $x_m \neq y_n$ 且 $z_k \neq y_n$, $\Rightarrow Z$ 是 X 和 Y_{n-1} 的一个 LCS;

(3) 与 (2) 对称, 类似可证。

综上, 定理 15.1 证毕。

2. 最优公共子序列的递归结构



用 $c[i][j]$ 记录序列和的最长公共子序列的长度。

其中, $X_i=\{x_1, x_2, \dots, x_i\}$; $Y_j=\{y_1, y_2, \dots, y_j\}$ 。

当 $i=0$ 或 $j=0$ 时, 空

列, 故此时 $C[i][j]$

其它情况可由最优

定理15.1将 X 和 Y 的LCS分解为2种情况:

(1) if $x_m = y_n$ then //解一个子问题

找 X_{m-1} 和 Y_{n-1} 的LCS;

(2) if $x_m \neq y_n$ then //解二个子问题

找 X_{m-1} 和 Y 的LCS; 找 X 和 Y_{n-1} 的LCS;

取两者中的最大的;

$$c[i][j] = \begin{cases} 0 & i = 0 \parallel j = 0 \\ c[i-1][j-1] + 1 & i, j > 0; x_i = y_j \\ \max\{c[i][j-1], c[i-1][j]\} & i, j > 0; x_i \neq y_j \end{cases}$$

3. 计算最优值



— 数据结构设计

$c[0..m, 0..n]$ // 存放最优解值, 计算时行优先
 $b[1..m, 1..n]$ // 解矩阵, 存放构造最优解信息

$b[i, j] = \begin{cases} \nwarrow & \text{如果 } c[i, j] \text{ 由 } c[i-1, j-1] \text{ 确定} \\ \uparrow & \text{如果 } c[i, j] \text{ 由 } c[i-1, j] \text{ 确定} \\ \leftarrow & \text{如果 } c[i, j] \text{ 由 } c[i, j-1] \text{ 确定} \end{cases}$

当构造解时, 从 $b[m, n]$ 出发, 上溯至 $i=0$ 或 $j=0$ 止
上溯过程中, 当 $b[i, j]$ 包含 " $\nwarrow$ " 时打印出 $x_i(y_j)$

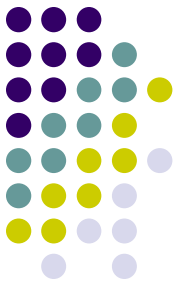


- 若 $X = \langle A, B, C, B, D, A, B \rangle$, $Y = \langle B, D, C, A, B, A \rangle$, 则

$i \quad j$	0	1	2	3	4	5	6	
y_j	x_i	B	D	C	A	B	A	
0	0	0	0	0	0	0	0	
1	A	0	$\uparrow$ 0	$\uparrow$ 0	$\uparrow$ 0	$\swarrow$ 1	$\leftarrow$ 1	$\swarrow$ 1
2	B	0	$\swarrow$ 1	$\leftarrow$ 1	$\leftarrow$ 1	$\uparrow$ 1	$\swarrow$ 2	$\leftarrow$ 2
3	C	0	$\uparrow$ 1	$\uparrow$ 1	$\swarrow$ 2	$\leftarrow$ 2	$\uparrow$ 2	$\uparrow$ 2
4	B	0	$\swarrow$ 1	$\uparrow$ 1	$\uparrow$ 2	$\uparrow$ 2	$\swarrow$ 3	$\leftarrow$ 3
5	D	0	$\uparrow$ 1	$\swarrow$ 2	$\uparrow$ 2	$\uparrow$ 2	$\uparrow$ 3	$\uparrow$ 3
6	A	0	$\uparrow$ 1	$\uparrow$ 2	$\uparrow$ 2	$\swarrow$ 3	$\uparrow$ 3	$\swarrow$ 4
7	B	0	$\swarrow$ 1	$\uparrow$ 2	$\uparrow$ 2	$\uparrow$ 3	$\swarrow$ 4	$\uparrow$ 4

- 最优解: BCAB 或 BCBA

4. 构造最长公共子序列

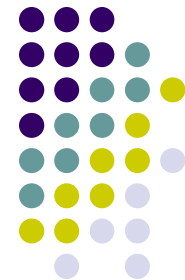


```
void LCS(int i, int j, char *x, int **b) {  
    if (i == 0 || j == 0) return;  
    if (b[i][j] == 1) {  
        LCS(i-1, j-1, x, b); cout<<x[i]; }  
    else if (b[i][j] == 2) LCS(i-1, j, x, b);  
    else LCS(i, j-1, x, b);  
}
```



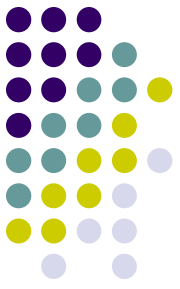

改进代码:

- 在算法LCS-Length和Print-LCS中, 可进一步将数组b省去。事实上, 数组元素 $c[i][j]$ 的值仅由 $c[i-1][j-1]$, $c[i-1][j]$ 和 $c[i][j-1]$ 这3个数组元素的值所确定。对于给定的数组元素 $c[i][j]$, 可以不借助于数组b而仅借助于c本身在常数时间内确定 $c[i][j]$ 的值是由 $c[i-1][j-1]$, $c[i-1][j]$ 和 $c[i][j-1]$ 中哪一个值所确定的。
- 如果只需要计算最长公共子序列的长度, 则算法的空间需求可大大减少。事实上, 在计算 $c[i][j]$ 时, 只用到数组c的第i行和第i-1行。因此, 用2行的数组空间就可以计算出最长公共子序列的长度。

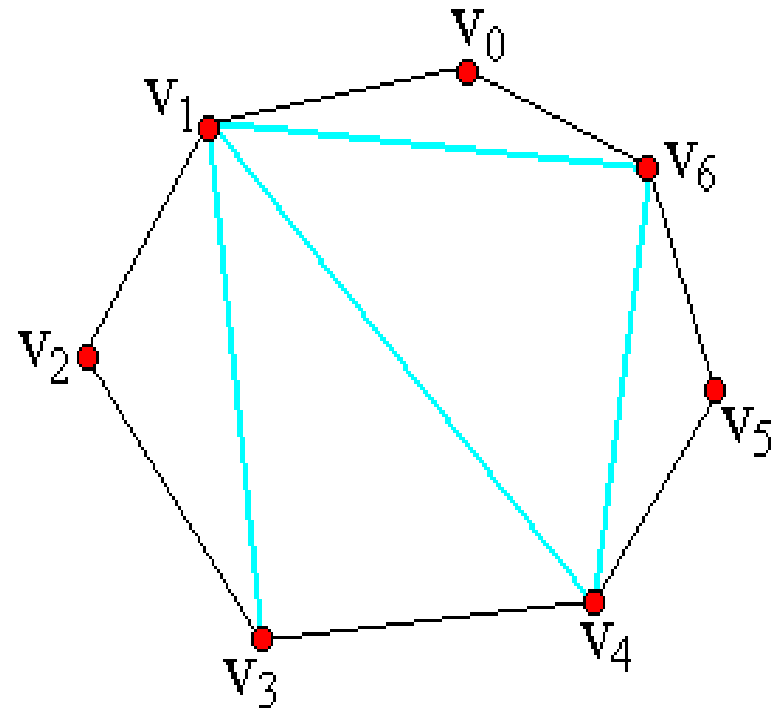
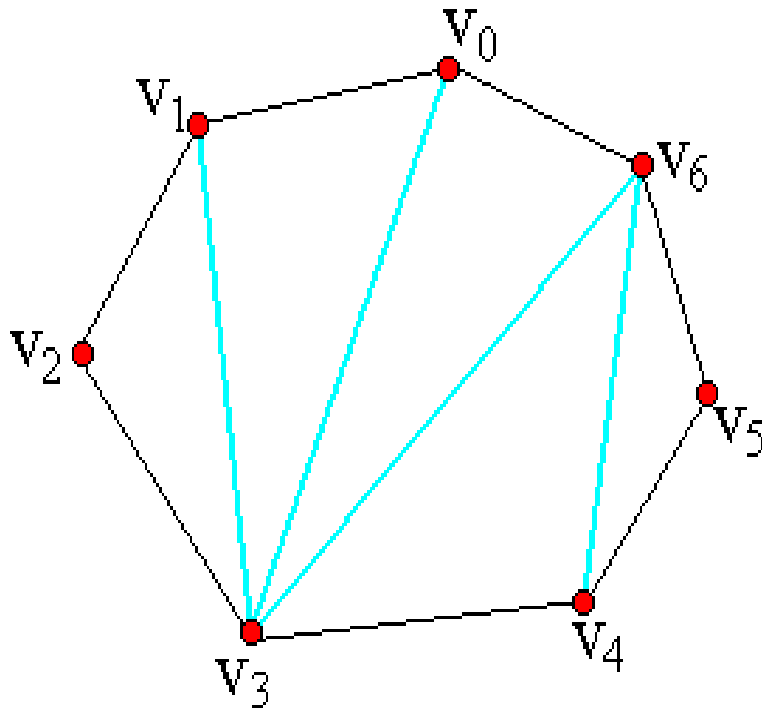


凸多边形最优三角剖分问题

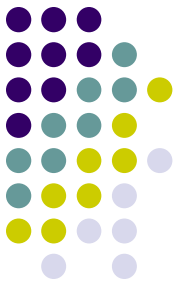
凸多边形最优三角剖分问题



- 用顶点的逆时针序列表示凸多边形，即



凸多边形最优三角剖分问题



- 问题描述:

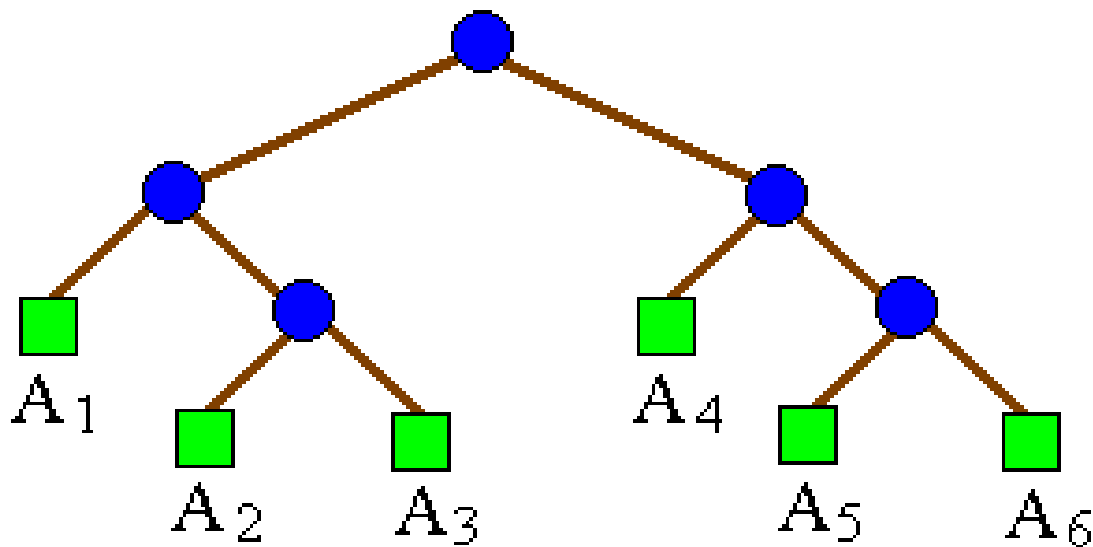
给定凸多边形 P ，以及定义在由多边形的边和弦组成的三角形上的权函数 w 。要求确定该凸多边形的三角剖分，使得该三角剖分所对应的权（即该三角剖分中诸三角形上的权）之和为最小。

权函数 w 可任意定义，如最小弦长等。

1. 三角剖分的结构及其相关问题



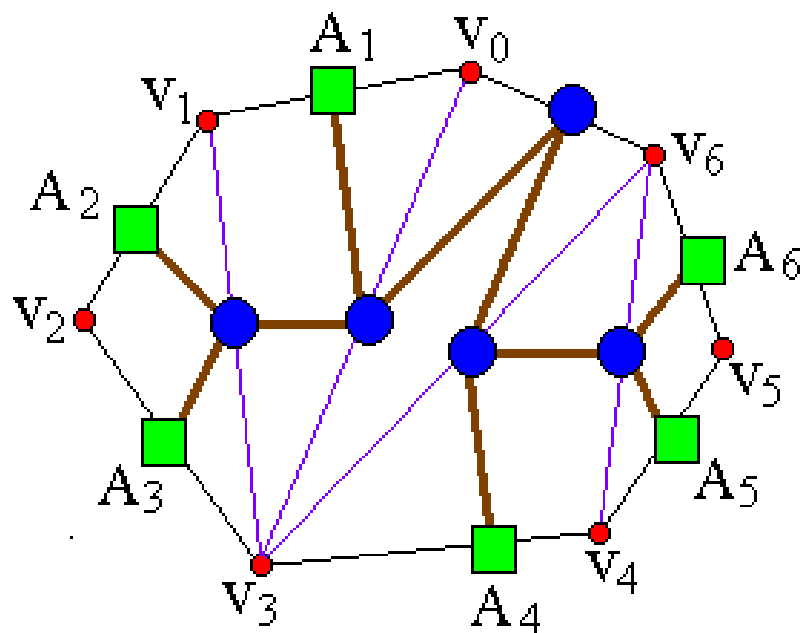
- 一个表达式的完全加括号方式相应于一棵完全二叉树，称为**表达式的语法树**。例如，完全加括号的矩阵连乘积 $((A_1(A_2A_3))(A_4(A_5A_6)))$ 所对应的语法树如图所示。



1. 三角剖分的结构及其相关问题



- 凸多边形 $\{v_0, v_1, \dots, v_{n-1}\}$ 的三角剖分也可以用语法树表示，**如**图所示。
- 矩阵连乘积中的每个矩阵 A_i 对应于凸 n 边形中的一条边 $v_{i-1}v_i$ 。三角剖分中的一条弦 $v_i v_j, i < j$ ，对应于矩阵连乘积 $A[i+1:j]$ 。



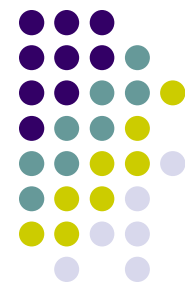
2. 最优子结构性质



- 凸多边形的最优三角剖分问题具有最优子结构性质。

事实上，若凸 $n+1$ 边形 $P=\{v_0, v_1, \dots, v_n\}$ 的最优三角剖分 T 包含三角形 $v_0v_kv_n$ ， $1 \leq k \leq n-1$ ，则 T 的权是3个部分权的和：三角形 $v_0v_kv_n$ 的权，子多边形 $\{v_0, v_1, \dots, v_k\}$ 和 $\{v_k, v_{k+1}, \dots, v_n\}$ 的权之和。可以断言，由 T 所确定的这2个子多边形的三角剖分也是最优的。因为若有 $\{v_0, v_1, \dots, v_k\}$ 或 $\{v_k, v_{k+1}, \dots, v_n\}$ 的更小权的三角剖分将导致 T 不是最优三角剖分的矛盾。

3. 最优三角剖分的递归结构



- 设 $t[i][j]$, $1 \leq i < j \leq n$, 为凸子多边形 $\{v_{i-1}, v_i, \dots, v_j\}$ 的最优三角剖分所对应的权函数值, 即最优值。
且设退化的多边形 $\{v_{i-1}, v_i\}$ 具有权值 0。
据此, 要计算的凸 $n+1$ 边形的最优值为 $t[1][n]$ 。
- $t[i][j]$ 的值可以利用最优子结构性质递归地计算:
当 $j-i \geq 1$ 时, 凸子多边形至少有 3 个顶点。
由最优子结构性质, $t[i][j]$ 的值应为 $t[i][k]$ 的值加上 $t[k+1][j]$ 的值, 再加上三角形 $v_{i-1}v_kv_j$ 的权值, 其中 $i \leq k \leq j-1$ 。

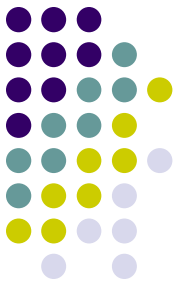
3. 最优三角剖分的递归结构



- 由于在计算时还不知道 k 的确切位置，而 k 的所有可能位置只有 $j-i$ 个，因此，可以在这 $j-i$ 个位置中选出使 $t[i][j]$ 值达到最小的位置。因此， $t[i][j]$ 可递归地定义为：

$$t[i][j] = \begin{cases} 0 & i = j \\ \min_{i \leq k < j} \{t[i][k] + t[k+1][j] + w(v_{i-1}v_kv_j)\} & i < j \end{cases}$$

4. 计算最优值



```
void MinWeightTriangulation(int n) {  
    for(r=2; r<n; r++) {  
        for(i=1; i<n-r+1; i++) {  
            j=i+r-1;  
            t[i][j]=t[i+1][j]+w(i-1,i,j);  
            s[i][j]=i;  
            for(k=i+1; k<j; k++) {  
                u=t[i][k]+t[k+1][j]+w(i-1,k,j);  
                if (u<t[i][j]) { t[i][j]=u; s[i][j]=k; }  
            }  
        }  
    }  
} // 算法的时间复杂度为 $O(n^3)$ , 空间复杂度为 $O(n^2)$ 。
```

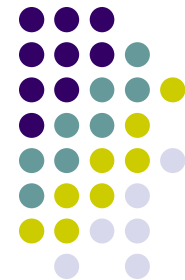
5. 构造最优三角剖分



/*按算法计算出的信息s指示的 $\triangle$ 第3个顶点的位置，输出最优三角剖分中的所有 $\triangle$ */

```
void Traceback(int i, int j) {  
    if (i==j) return;  
    Traceback(i, s[i][j]);  
    cout<<" $\triangle A$ "<<i<<"-A"<<s[i][j]+1;  
    cout<<"-A"<<j+1<<endl;  
    Traceback(s[i][j]+1, j);  
}
```





多边形游戏问题

多边形游戏问题



多边形游戏是一个单人玩的游戏，开始时有一个由 n 个顶点构成的多边形。每个顶点被赋予一个整数值，每条边被赋予一个运算符“+”或“*”。所有边依次用整数从1到 n 编号。

游戏第1步，将一条边删除。

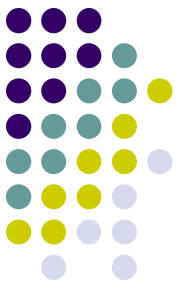
随后 $n-1$ 步按以下方式操作：

- (1) 选择一条边 E 以及由 E 连接着的2个顶点 $V1$ 和 $V2$ ；
- (2) 用一个新的顶点取代边 E 以及由 E 连接着的2个顶点 $V1$ 和 $V2$ 。将由顶点 $V1$ 和 $V2$ 的整数值通过边 E 上的运算得到的结果赋予新顶点。

最后，所有边都被删除，游戏结束。

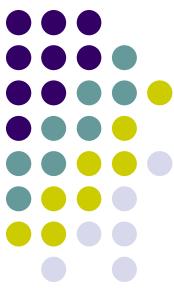
游戏的得分就是所剩顶点上的整数值。

问题：对于给定的多边形，计算最高得分。



1. 最优子结构性质

- 设多边形的顶点和边的顺时针的序列为 $op[1], v[1], op[2], v[2], \dots, op[n], v[n]$
- 在所给多边形中，从顶点 $i (1 \leq i \leq n)$ 开始，长度为 j (链中有 j 个顶点) 的顺时针链 $p(i, j)$ 可表示为 $v[i], op[i+1], \dots, v[i+j-1]$ 。
- 如果这条链的最后一次合并运算在 $op[i+s]$ 处发生 ($1 \leq s \leq j-1$)，则可在 $op[i+s]$ 处将链分割为2个子链 $p(i, s)$ 和 $p(i+s, j-s)$ 。



1. 最优子结构性质

- 设 $m1$ 是对子链 $p(i, s)$ 的任意一种合并方式得到的值，而 a 和 b 分别是在所有可能的合并中得到的最小值和最大值。 $m2$ 是 $p(i+s, j-s)$ 的任意一种合并方式得到的值，而 c 和 d 分别是在所有可能的合并中得到的最小值和最大值。

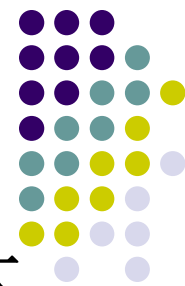
依此定义有 $a \leq m1 \leq b$, $c \leq m2 \leq d$

(1) 当 $op[i+s]='+'$ 时，显然有 $a+c \leq m \leq b+d$

(2) 当 $op[i+s]='*'$ 时，有 $\min\{ac, ad, bc, bd\} \leq m \leq \max\{ac, ad, bc, bd\}$

- 换句话说，主链的最大值和最小值可由子链的最大值和最小值得到——满足最优子结构性质。

2. 多边形游戏的递归求解



为了求链合并的最大值，必须同时求子链合并的最大值和最小值。

设 $m[i,j,0]$ 是链 $p(i,j)$ 合并的最小值， $m[i,j,1]$ 是最大值。

又设最优合并在 $op[i+s]$ 处将 $p(i,j)$ 分为2个长度小于 j 的子链 $p(i,i+s)$ 和 $p(i+s,j-s)$ ，且从顶点 i 开始的长度小于 j 的子链的最大值和最小值均已计算出。

为叙述方便，记 $a=m[i,i+s,0]$ ， $b=m[i,i+s,1]$ ，
 $c=m[i+s,j-s,0]$ ， $d=m[i+s,j-s,1]$

(1)当 $op[i+s]='+'$ 时， $m[i,j,0]=a+c$ ， $m[i,j,1]=b+d$

2. 多边形游戏的递归求解



(2) 当 $op[i+s]='*'$ 时,

$m[i, j, 0] = \min\{ac, ad, bc, bd\},$

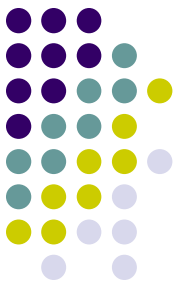
$m[i, j, 1] = \max\{ac, ad, bc, bd\}$

综合(1)和(2), 将 $p(i, j)$ 在 $op[i+s]$ 处断开的最大值为 $\max f(i, j, s)$, 最小值为 $\min f(i, j, s)$, 则

$$\min f(i, j, s) = \begin{cases} a + c & op[i+s] = '+' \\ \min\{ac, ad, bc, bd\} & op[i+s] = '*' \end{cases}$$

$$\max f(i, j, s) = \begin{cases} b + d & op[i+s] = '+' \\ \max\{ac, ad, bc, bd\} & op[i+s] = '*' \end{cases}$$

2. 多边形游戏的递归求解



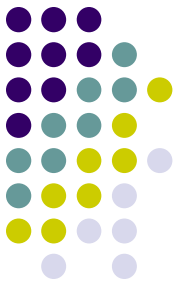
由于最优断开位置 s 有 $1 \leq s \leq j-1$ 的 $j-1$ 种情况，由此可知，

$$m[i, j, 0] = \min_{1 \leq s \leq j} \{ \min f(i, j, s) \}, 1 \leq i, j \leq n$$

$$m[i, j, 1] = \max_{1 \leq s \leq j} \{ \max f(i, j, s) \}, 1 \leq i, j \leq n$$

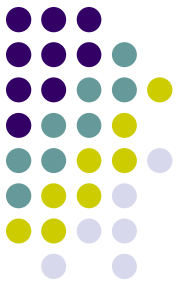
初始边界值显然为 $m[i, 1, 0] = v[i]$, $m[i, 1, 1] = v[i]$, $1 \leq i \leq n$ 。

3. 多边形游戏的算法描述

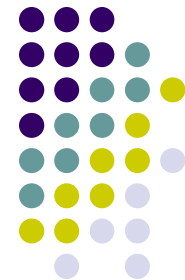


```
void MinMax(int i, int j, int k) {  
    int e[4], l, a = m[i][k][0], b = m[i][k][1],  
    r = (i + k + 1) % N, c = m[r][j - k - 1][0],  
    d = m[r][j - k - 1][1];  
    if (op[(r - 1 + N) % N] == '+') {  
        minf = a + c; maxf = b + d;  
    } else {  
        e[0] = a * c; e[1] = a * d; e[2] = b * c;  
        e[3] = b * d; minf = e[0]; maxf = e[0];  
        for (l = 1; l < 4; l++) {  
            if (minf > e[l]) minf = e[l];  
            if (maxf < e[l]) maxf = e[l];  
        }  
    }  
}
```


3. 多边形游戏的算法描述

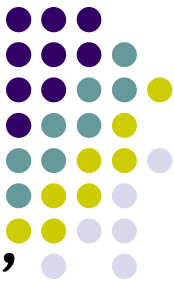


```
void PolyMax() {
    int i, j, k, max;
    for (j = 1; j < N; j++)
        for (i = 0; i < N; i++)
            for (k = 0; k < j; k++) {
                MinMax(i, j, k);
                if (m[i][j][0] > minf) m[i][j][0] = minf;
                if (m[i][j][1] < maxf) m[i][j][1] = maxf;
            }
        max = m[0][N - 1][1];
        for (i = 1; i < N; i++)
            if (max < m[i][N - 1][1]) max = m[i][N - 1][1];
    printf("%d\n", max);
} // 多边形游戏的算法时间复杂度为 $O(n^3)$ 。
```



图像压缩问题

图像压缩问题



通常用像素点灰度值序列 $\{p_1, p_2, \dots, p_n\}$ 表示图像，其中， $0 \leq p_i \leq 255$ ，可用8位表示1个像素。

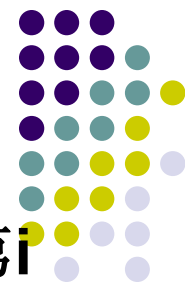
图像的变位压缩存储格式将所给的像素点序列 $\{p_1, p_2, \dots, p_n\}$ 分割成 m 个连续段 $S_1, S_2, \dots, S_m$ 。第 i 个像素段 S_i 中有 $l[i]$ 个像素，且该段中每个像素都只用 $b[i]$ 位表示。

设 $t[i] = \sum_{k=1}^{i-1} l[k]$ ，则第 i 个像素段 S_i 为 $S_i = \{p_{t[i]+1}, \dots, p_{t[i]+l[i]}\}$

设 $h_i = \left\lceil \log \left(\max_{t[i]+1 \leq k \leq t[i]+l[i]} p_k + 1 \right) \right\rceil$ ，则 $h_i \leq b[i] \leq 8$ 。

因此需要用3位表示 $b[i]$ 。

图像压缩问题



如果限制 $1 \leq l[i] \leq 255$ ，则需要用8位表示 $l[i]$ 。因此，第 i 个像素段所需的存储空间为 $l[i] * b[i] + 11$ 位。

按此格式存储像素序列 $\{p_1, p_2, \dots, p_n\}$,

需要 $\sum_{i=1}^m l[i] * b[i] + 11m$ 位的存储空间。

图象压缩问题描述： 要求确定像素序列 $\{p_1, p_2, \dots, p_n\}$ 的最优分段，使得依此分段所需的存储空间最少。
每个分段的长度不超过256位。

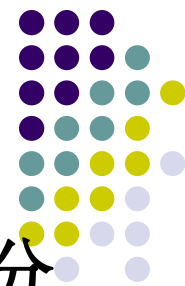
1. 最优子结构性质



设 $l[i]$, $b[i]$, $1 \leq i \leq m$, 是 $\{p_1, p_2, \dots, p_n\}$ 的一个最优分段。显而易见, $l[1]$, $b[1]$ 是 $\{p_1, \dots, p_{l[1]}\}$ 的一个最优分段, 且 $l[i]$, $b[i]$, $2 \leq i \leq m$, 是 $\{p_{l[1]+1}, \dots, p_n\}$ 的最优分段。

→ 图象压缩问题满足最优子结构性质。

2. 图像压缩问题的递归结构



设 $s[i]$, $1 \leq i \leq n$, 是像素序列 $\{p_1, \dots, p_n\}$ 的最优分段所需的存储位数, 则

$$s[i] = \min_{1 \leq k \leq \min\{i, 256\}} \{s[i-k] + k * b_{\max(i-k+1, i)}\} + 11$$

其中

$$b_{\max(i, j)} = \left\lceil \log \left(\max_{i \leq k \leq j} \{p_k\} + 1 \right) \right\rceil$$

3. 计算最优值



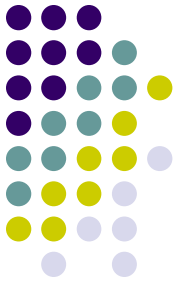
```
void Compress(int n, int p[ ]) {  
    int Lmax=256, header=11;  
    s[0]=0;  
    for(int i=1; i<=n; i++) {  
        b[i]=length(p[i]); int bmax=b[i];  
        s[i]=s[i-1]+bmax; l[i]=1;  
        for(int j=2; j<=i && j<=Lmax; j++) {  
            if(bmax<b[i-j+1]) bmax=b[i-j+1];  
            if(s[i]>s[i-j]+j*bmax)  
                { s[i]=s[i-j]+j*bmax; l[i]=j; }  
        }  
        s[i]+=header;  
    }  
} // 算法的计算时间为O(n)
```

4. 构造最优解



```
void Traceback(int n, int &i) {  
    if (n==0) return;  
    Traceback(n-l[n], i);  
    s[i++]=n-l[n];  
}
```

```
void Output(int n) {  
    cout<<"The optimal value is "<<s[n]<<endl;  
    int m=0;  
    Traceback(n, m);  
    s[m]=n;  
    cout<<"Decompose into "<<m<<" segments:"<<endl;  
    for(int j=1; j<=m; j++) { l[j]=l[s[j]]; b[j]=b[s[j]]; }  
    for(j=1; j<=m; j++) cout<<l[j]<<'\\t'<<b[j]<<endl;  
} // 算法的计算时间为O(n)
```

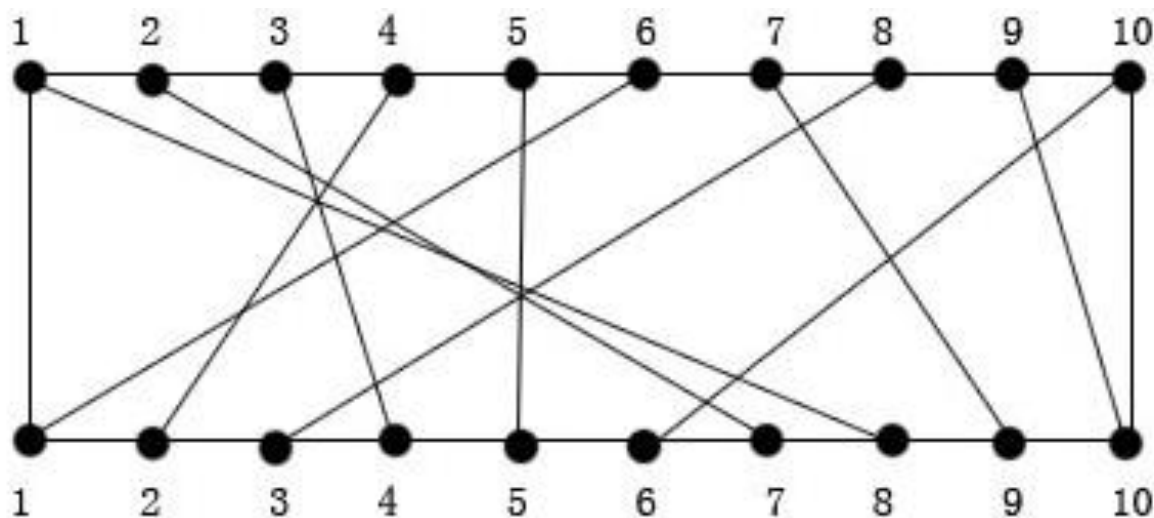



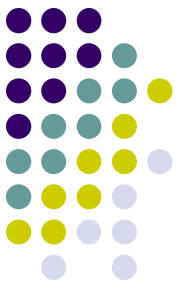
电路布线

电路布线问题



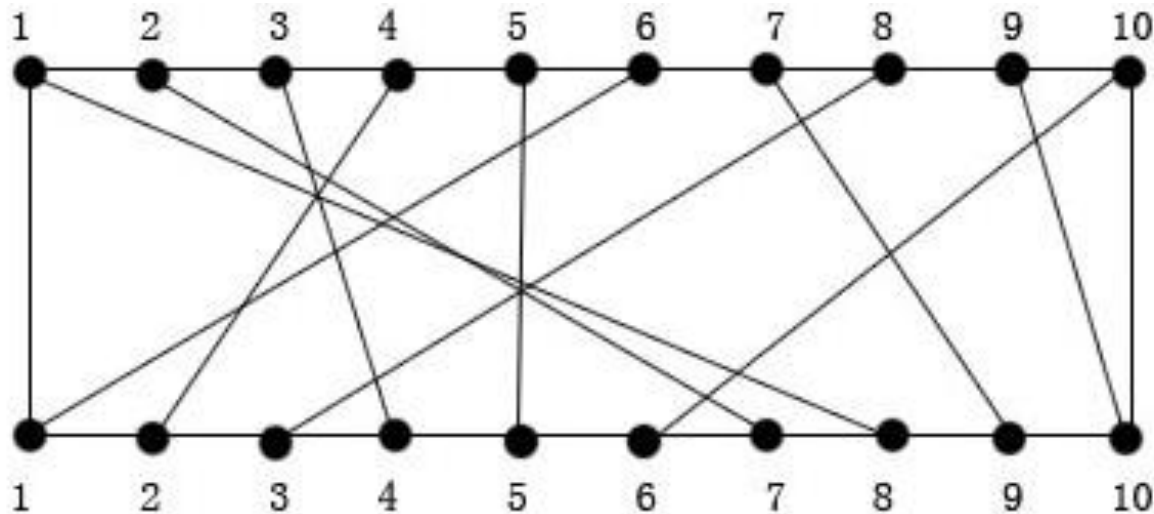
在一块电路板的上、下两端分别有 n 个接线柱。根据电路设计，要求用导线 $(i, \pi(i))$ 将上端接线柱 i 与下端接线柱 $\pi(i)$ 相连，如下图。其中， $\pi(i), 1 \leq i \leq n$, 是 $\{1, 2, \dots, n\}$ 的一个排列。导线 $(i, \pi(i))$ 称为该电路板上的第 i 条连线。对于任何 $1 \leq i \leq j \leq n$, 第 i 条连线和第 j 条连线相交的充要条件是 $\pi(i) > \pi(j)$.

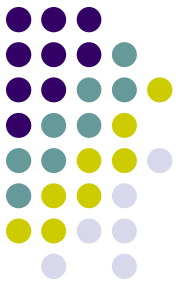




电路布线问题

- 在制作电路板时，要求将这 n 条线分布到若干个绝缘层上，在同一层上的连线不能相交。
- 电路布线问题要确定将哪些连线安排在第一层上，使得该层上有尽可能多的连线。换句话说，该问题要求确定导线集 $\text{Nets} = \{ (i, \pi(i)), 1 \leq i \leq n \}$ 的最大不相交子集。





1.最优子结构性质

- 记 $N(i,j) = \{t | (t, \pi(i)) \in \text{Nets}, t \leq i, \pi(t) \leq j\}$. $N(i,j)$ 的最大不相交子集为 $\text{MNS}(i,j)$ 。 $\text{Size}(i,j) = |\text{MNS}(i,j)|$ 。

1) 当 $i = 1$ 时

$$\text{MNS}(1,j) = N(1,j) = \begin{cases} \text{空} & j < \pi(1) \\ \{(1, \pi(1))\} & j \geq \pi(1) \end{cases}$$

2)当 $i > 1$ 时



① $j < \pi(i)$ 。此时， $(i, \pi(i))$ 不属于 $N(i, j)$ 。故在这种情况下，
 $N(i, j) = N(i-1, j)$ ，从而 $\text{Size}(i, j) = \text{Size}(i-1, j)$ 。

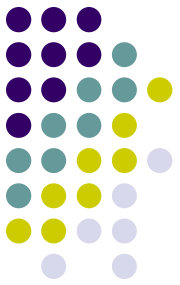
② $j \geq \pi(i)$ 。此时，若 $(i, \pi(i)) \in \text{MNS}(i, j)$ ，则对任意
 $(t, \pi(t)) \in \text{MNS}(i, j)$ 有 $t < i$ 且 $\pi(t) < \pi(i)$ ；

否则， $(t, \pi(t))$ 与 $(i, \pi(i))$ 相交。在这种情况下 $\text{MNS}(i, j) - \{(i, \pi(i))\}$
是 $N(i-1, \pi(i)-1)$ 的最大不相交子集。否则，子集 $\text{MNS}(i-1, \pi(i)-1) \cup \{(i, \pi(i))\}$ 包含于 $N(i, j)$ 是比 $\text{MNS}(i, j)$ 更大的 $N(i, j)$ 的不相交子集。这与 $\text{MNS}(i, j)$ 的定义相矛盾。

若 $(i, \pi(i))$ 不属于 $\text{MNS}(i, j)$ ，则对任意 $(t, \pi(t)) \in \text{MNS}(i, j)$ ，有 $t < i$ 。
从而 $\text{MNS}(i, j)$ 包含于 $N(i-1, j)$ ，因此， $\text{Size}(i, j) \leq \text{Size}(i-1, j)$ 。

另一方面， $\text{MNS}(i-1, j)$ 包含于 $N(i, j)$ ，故又有 $\text{Size}(i, j) \geq \text{Size}(i-1, j)$ ，
从而 $\text{Size}(i, j) = \text{Size}(i-1, j)$ 。

2. 递归计算最优值



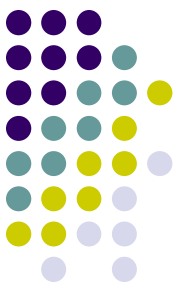
- 经以上后分，可电路布线问题的最优值为 $\text{Size}(n,n)$ 。由该问题的最优子结构性质可知：

1) 当 $j=1$ 时，

$$\text{Size}(1,j) = \begin{cases} 0 & j < \pi(1) \\ 1 & j \geq \pi(1) \end{cases}$$

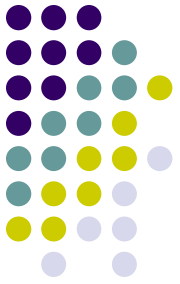
2) 当 $j > 1$ 时，

$$\text{Size}(i,j) = \begin{cases} \text{Size}(i-1,j) & j < \pi(i) \\ \max\{\text{Size}(i-1,j), \text{Size}(i-1, \pi(i)-1)+1\} & j \geq \pi(i) \end{cases}$$



3. 根据上面递归式可以得到二维表:

i \ j	1	2	3	4	5	6	7	8	9	10
1	0	0	0	0	0	0	0	1	1	1
2	0	0	0	0	0	0	1	1	1	1
3	0	0	0	1	1	1	1	1	1	1
4	0	1	1	1	1	1	1	1	1	1
5	0	1	1	1	2	2	2	2	2	2
6	1	1	1	1	2	2	2	2	2	2
7	1	1	1	1	2	2	2	2	3	3
8	1	1	2	2	2	2	2	2	3	3
9	1	1	2	2	2	2	2	2	3	4
10	1	1	2	2	2	2	2	2	3	4



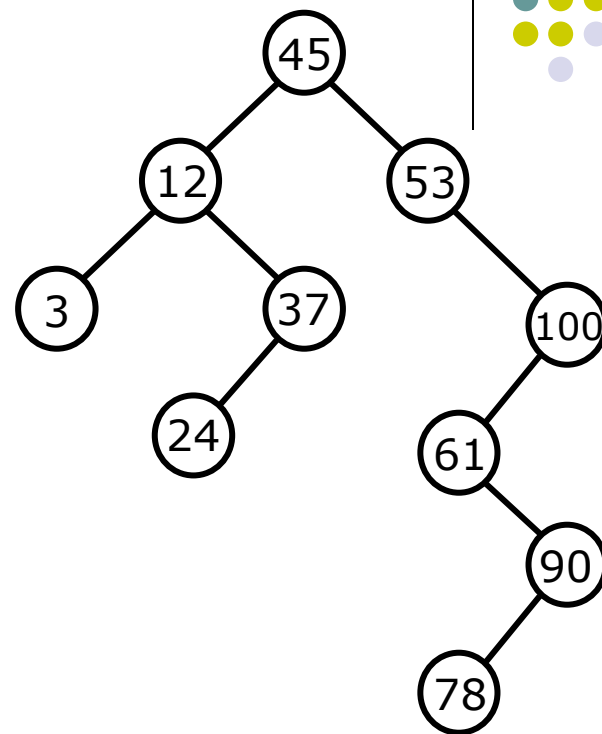
最优二叉查找树

最优二叉查找树



- 二叉查找树

- 1) 若左子树不空，则左子树上所有节点的值均小于它的根节点的值；
- 2) 若右子树不空，则右子树上所有节点的值均大于它的根节点的值；
- 3) 左、右子树也分别为二叉查找树



在随机的情况下，二叉查找树的平均查找长度为 $O(\log n)$ 。

最优二叉查找树



设 $\{r_1, r_2, \dots, r_n\}$ 是 n 个记录的集合，其查找概率分别是 $\{p_1, p_2, \dots, p_n\}$ ，最优二叉查找树是以这 n 个记录构成的二叉查找树中具有最少平均比较次数的二叉查找树，即

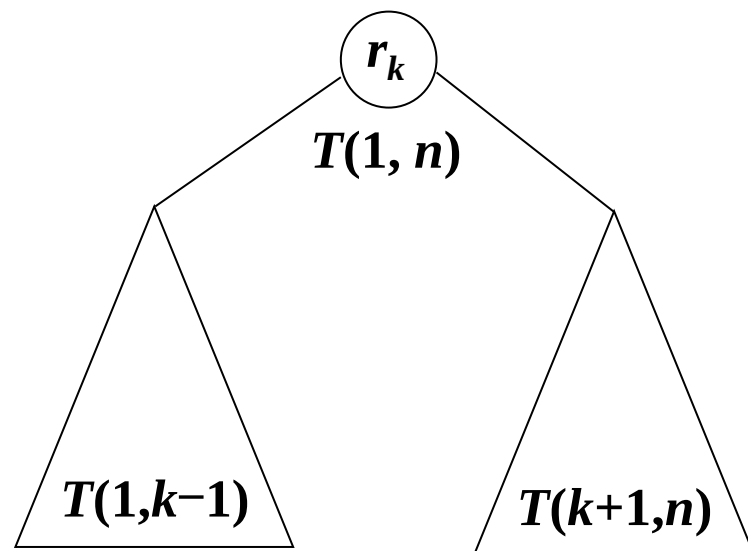
$$\sum_{i=1}^n p_i \times c_i$$

最小，其中 p_i 是记录 r_i 的查找概率， c_i 是在二叉查找树中查找 r_i 的比较次数。

证明最优二叉查找树满足最优性原理

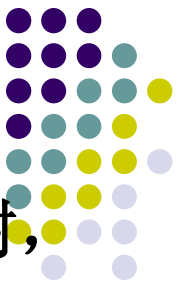
将由 $\{r_1, r_2, \dots, r_n\}$ 构成的二叉查找树记为 $T(1, n)$ ，其中 r_k ($1 \leq k \leq n$) 是 $T(1, n)$ 的根结点，则其左子树 $T(1, k-1)$ 由 $\{r_1, \dots, r_{k-1}\}$ 构成，其右子树 $T(k+1, n)$ 由 $\{r_{k+1}, \dots, r_n\}$ 构成。

若 $T(1, n)$ 是最优二叉查找树，则其左子树 $T(1, k-1)$ 和右子树 $T(k+1, n)$ 也是最优二叉查找树，如若不然，假设 $T'(1, k-1)$ 是比 $T(1, k-1)$ 更优的二叉查找树，则 $T'(1, k-1)$ 的平均比较次数小于 $T(1, k-1)$ 的平均比较次数，从而由 $T'(1, k-1)$ 、 r_k 和 $T(k+1, n)$ 构成的二叉查找树 $T'(1, n)$ 的平均比较次数小于 $T(1, n)$ 的平均比较次数，这与 $T(1, n)$ 是最优二叉查找树的假设相矛盾。



以 r_k 为根的二叉查找树

最优二叉查找树



设 $T(i, j)$ 是由记录 $\{r_i, \dots, r_j\} (1 \leq i \leq j \leq n)$ 构成的二叉查找树， $C(i, j)$ 是这棵二叉查找树的平均比较次数。虽然最后的结果是 $C(1, n)$ ，但遵循动态规划法的求解方法，需要求出所有较小子问题 $C(i, j)$ 的值。考虑从 $\{r_i, \dots, r_j\}$ 中选择一个记录 r_k 作为二叉查找树的根结点，可以得到如下关系：

$$\begin{aligned} C(i, j) &= \min_{i \leq k \leq j} \{ p_k \times 1 + \sum_{s=i}^{k-1} p_s \times (r_s \text{ 在 } T(i, k-1) \text{ 中的层数} + 1) + \sum_{s=k+1}^j p_s \times (r_s \text{ 在 } T(k+1, n) \text{ 中的层数} + 1) \} \\ &= \min_{i \leq k \leq j} \{ p_k + \sum_{s=i}^{k-1} p_s \times r_s \text{ 在 } T(i, k-1) \text{ 中的层数} + \sum_{s=i}^{k-1} p_s + \sum_{s=k+1}^j p_s \times r_s \text{ 在 } T(k+1, n) \text{ 中的层数} + \sum_{s=k+1}^j p_s \} \\ &= \min_{i \leq k \leq j} \{ \sum_{s=i}^{k-1} p_s \times r_s \text{ 在 } T(i, k-1) \text{ 中的层数} + \sum_{s=k+1}^j p_s \times r_s \text{ 在 } T(k+1, n) \text{ 中的层数} + \sum_{s=i}^j p_s \} \\ &= \min_{i \leq k \leq j} \{ C(i, k-1) + C(k+1, j) + \sum_{s=i}^j p_s \} \end{aligned}$$

最优二叉查找树

因此，得到如下动态规划函数：

$$C(i, i-1)=0 \quad (1 \leq i \leq n+1)$$

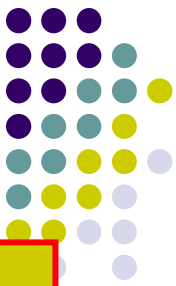
$$C(i, i)=p_i \quad (1 \leq i \leq n)$$

$$C(i, j)=\min\{C(i, k-1)+C(k+1, j)+\sum_{s=i}^j p_s\} \quad (1 \leq i \leq j \leq n, i \leq k \leq j)$$

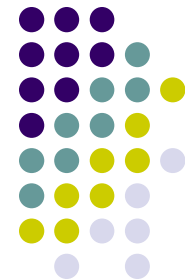
设一个二维表 $C[n+1][n+1]$ ，其中 $C[i][j]$ 表示二叉查找树 $T(i, j)$ 的平均比较次数。

当 $k=1$ 时，求 $C[i][j]$ 需要用到 $C[i][0]$ ，当 $k=n$ 时，求 $C[i][j]$ 需要用到 $C[n+1][j]$ ，所以，二维表 $C[n+1][n+1]$ 行下标的范围为 $1 \sim n+1$ ，列下标的范围为 $0 \sim n$ 。

为了在求出由 $\{r_1, r_2, \dots, r_n\}$ 构成的二叉查找树的平均比较次数的同时得到最优二叉查找树，设一个二维表 $R[n+1][n+1]$ ，其下标范围与二维表 C 相同， $R[i][j]$ 表示二叉查找树 $T(i, j)$ 的根结点的序号。



最优二叉查找树



例如，集合 $\{A, B, C, D\}$ 的查找概率是 $\{0.1, 0.2, 0.4, 0.3\}$ ，二维表C和R的初始情况如图所示。

	0	1	2	3	4
1	0	0.1			
2		0	0.2		
3			0	0.4	
4				0	0.3
5					0

	0	1	2	3	4
1		1			
2			2		
3				3	
4					4
5					

最优二叉查找树



在二维表**C**和**R**中只需计算主对角线以上的元素。

首先计算 **$C(1, 2)$** :

$$C(1,2) = \min \left\{ \begin{array}{l} k=1: C(1,0) + C(2,2) + \sum_{s=1}^2 p_s = 0 + 0.2 + 0.3 = 0.5 \\ k=2: C(1,1) + C(3,2) + \sum_{s=1}^2 p_s = 0.1 + 0 + 0.3 = 0.4 \end{array} \right\} = 0.4$$

在前两个记录构成的最优二叉查找树的根结点的序号是**2**。

最优二叉查找树



按对角线逐条计算每一个 $C(i, j)$ 和 $R(i, j)$ ，得到最终表。

	0	1	2	3	4
1	0	0.1	0.4	1.1	1.7
2		0	0.2	0.8	1.4
3			0	0.4	1.0
4				0	0.3
5					0

$d=3$
 $d=2$
 $d=1$

(a) 二维表C

	0	1	2	3	4
1		1	2	3	3
2			2	3	3
3				3	3
4					4
5					

$d=3$
 $d=2$
 $d=1$

(b) 二维表R

最终表的状态

最优二叉查找树



设 n 个字符的查找概率存储在数组 $p[n]$ 中，动态规划法求解最优二叉查找树的算法如下：

```
// 最优二叉查找树算法
```

```
double OptimalBST(int n, double p[ ], double C[ ][ ], int R[ ][ ] )
```

```
{
```

```
    for (i=1; i<=n; i++) //初始化
```

```
    {
```

```
        C[i][i-1]=0;
```

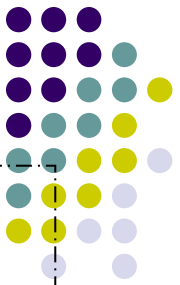
```
        C[i][i]=p[i];
```

```
        R[i][i]=i;
```

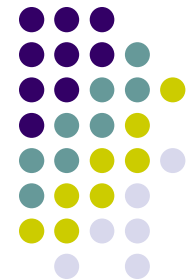
```
    }
```

```
    C[n+1][n]=0;
```

最优二叉查找树



```
for (d=1; d<n; d++)    //按对角线逐条计算
    for (i=1; i<=n-d; i++)
    {
        j=i+d; min= $\infty$ ; mink=i; sum=0;
        for (k=i; k<=j; k++)
        {
            sum=sum+p[k];
            if (C[i][k-1]+C[k+1][j]<min) {
                min=C[i][k-1]+C[k+1][j];
                mink=k;
            }
        }
        C[i][j]=min+sum;
        R[i][j]=mink;
    }
return C[1][n];
}
```



0-1背包问题

0-1背包问题



3.1

背包问题

背包问题: 从 n 个物件(每个物件的体积为 $W_i, i=1, 2, \dots, n$)中选取若干个恰好能够填满体积为 T 的背包。例如,

W_1	$W_2 = 2$	$W_3 = 3$	$n =$	6				
W_4	$W_5 = 4$	$W_6 = 8$	$T =$	15				
$= 2$								

0-1背包问题



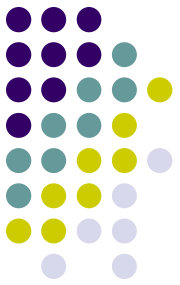
3.1

背包问题

用布尔函数表示背包问题的解:

$$K(T, n) = \begin{cases} 1, & T=0 \\ 0, & T<0 \\ 0, & T>0 \text{ 且 } n<1 \\ K(T, n-1) \text{ 或 } K(T-w_n, n-1), & T>0 \text{ 且 } n\geq 1 \end{cases}$$

0-1背包问题



```
int Knapsack(int n, int T) {  
    if (T==0) return 1;  
    else {  
        if (T<0 || n<1) return 0;  
        else {  
            if (Knapsack(n-1,T-W[n-1])!=1) {  
                printf(n,W[n-1]);  
                return 1;  
            }  
            else return Knapsack(n-1,T);  
        }  
    }  
}  
} //Knapsack递归算法
```

0-1背包问题



问题描述： 给定 n 种物品和一个背包。物品 i 的重量为 w_i ，价值为 v_i ，背包的容量(能够承载的重量)为 C 。问应如何选择装入背包的物品，使得装入背包中物品的总价值最大？

0-1背包问题是一个特殊的整数规划问题。

$$\begin{aligned} & \max \sum_{i=1}^n v_i x_i \\ & \begin{cases} \sum_{i=1}^n w_i x_i \leq C \\ x_i \in \{0,1\}, 1 \leq i \leq n \end{cases} \end{aligned}$$

0-1背包问题



1. 最优子结构: 设 $(y_1, y_2, \dots, y_n)$ 是所给0-1背包问题的一个最优解, 则 $(y_2, \dots, y_n)$ 是下列子问题的一个最优解:

$$\begin{cases} \max \sum_{i=2}^n v_i x_i \\ \sum_{i=2}^n w_i x_i \leq c - w_1 y_1 \\ x_i, y_1 \in \{0, 1\}, 2 \leq i \leq n \end{cases}$$

反证法: 设 $(z_2, \dots, z_n)$ 是上述子问题的一个最优解, 则

$$\begin{aligned} \sum_{i=1}^n v_i y_i &< \sum_{i=2}^n v_i z_i + v_1 y_1 \\ \sum_{i=2}^n w_i z_i &\leq c - w_1 y_1 \end{aligned}$$

矛盾。因此, **0-1**背包问题具有最优子结构性质。

0-1背包问题



2. 递归关系： 设所给**0-1**背包问题的子问题

$$\max \sum_{k=i}^n v_k x_k$$

$$\begin{cases} \sum_{k=i}^n w_k x_k \leq j \\ x_k \in \{0,1\}, i \leq k \leq n \end{cases}$$

的最优值为 **$m(i, j)$** ，即 **$m(i, j)$** 是背包容量为 **j** ，可选择物品为 **$i, i+1, \dots, n$** 时 **0-1**背包问题的最优值。

0-1背包问题



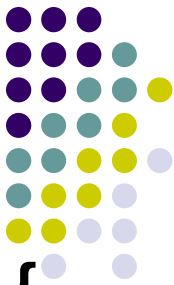
2. 递归关系:

由**0-1**背包问题的最优子结构性质, 可以建立计算 $m(i, j)$ 的递归式如下:

$$m(i, j) = \begin{cases} \max\{m(i+1, j), m(i+1, j-w_i) + v_i\} & j \geq w_i \\ m(i+1, j) & 0 \leq j < w_i \end{cases}$$

$$m(n, j) = \begin{cases} v_n & j \geq w_n \\ 0 & 0 \leq j < w_n \end{cases}$$

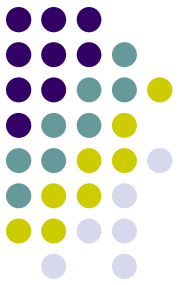
0-1背包问题



3. 算法描述:

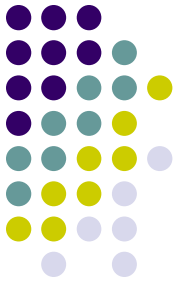
```
void Knapsack(Type v, int w, int c, int n, Type **m) {  
    int jMax=min(w[n]-1,c);  
    for(j=w[n]; j<=c; j++) m[n][j]=v[n];  
    for(int i=n-1; i>1; i--) {  
        jMax=min(w[i]-1,c);  
        for(j=0; j<=jMax; j++) m[i][j]=m[i+1][j];  
        for(j=w[i]; j<=c; j++)  
            m[i][j]=max(m[i+1][j],m[i+1][j-w[i]]+v[i]);  
    }  
    m[1][c]=m[2][c];  
    if(c>=w[1]) m[1][c]=max(m[1][c],m[2][c-w[1]]+v[1]);  
} // 算法的时间复杂度为O(nc)
```

0-1背包问题



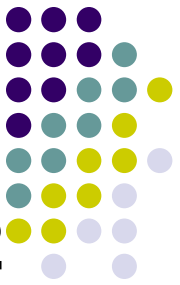
4. 构造最优解:

```
void Traceback(int c, int n, int x[ ]) {  
    for (int i=1; i<n; i++)  
        if (m[i][c]==m[i+1][c]) x[i]=0;  
        else  
        {  
            x[i]=1;  
            c-=w[i];  
        }  
    x[n]=m[n][c]>0 ? 1 : 0;  
}
```



流水作业调度

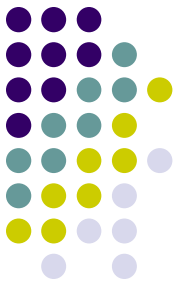
流水作业调度



n 个作业 $\{1, 2, \dots, n\}$ 要在由2台机器M1和M2组成的流水线上完成加工。每个作业加工的顺序都是先在M1上加工，然后在M2上加工。M1和M2加工作业 i 所需的时间分别为 a_i 和 b_i 。

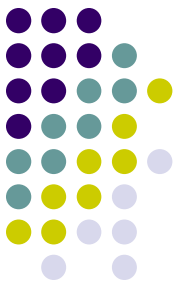
流水作业调度问题描述：要求确定这 n 个作业的最优加工顺序，使得从第一个作业在机器M1上开始加工，到最后一个作业在机器M2上加工完成所需的时间最少。

流水作业调度



分析:

- 直观上，最优调度应使机器**M1**没有空闲时间，且机器**M2**的空闲时间最少。在一般情况下，机器**M2**上会有机器空闲和作业积压2种情况。
- 设全部作业的集合为 $N=\{1, 2, \dots, n\}$ 。 $S \subseteq N$ 是 N 的作业子集。在一般情况下，机器**M1**开始加工**S**中作业时，机器**M2**还在加工其它作业，要等时间**t**后才可利用。将这种情况下完成**S**中作业所需的最短时间记为 $T(S, t)$ 。流水作业调度问题的最优值为 $T(N, 0)$ 。

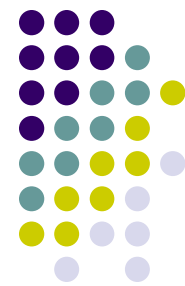


1. 最优子结构性质

设 π 是所给 n 个流水作业的一个最优调度，它所需的加工时间为 $a_{\pi(1)}+T'$ 。其中 T' 是在机器 M_2 的等待时间为 $b_{\pi(1)}$ 时，安排作业 $\pi(2), \dots, \pi(n)$ 所需的时间。记 $S=N-\{\pi(1)\}$ ，则有 $T'=T(S, b_{\pi(1)})$ 。

证明：由 T 的定义知 $T' \geq T(S, b_{\pi(1)})$ 。若 $T' > T(S, b_{\pi(1)})$ ，设 π' 是作业集 S 在机器 M_2 的等待时间为 $b_{\pi(1)}$ 情况下的一个最优调度。则 $\pi(1), \pi'(2), \dots, \pi'(n)$ 是 N 的一个调度，且该调度所需的时间为 $a_{\pi(1)}+T(S, b_{\pi(1)}) < a_{\pi(1)}+T'$ 。这与 π 是 N 的最优调度矛盾。故 $T' \leq T(S, b_{\pi(1)})$ 。从而 $T'=T(S, b_{\pi(1)})$ 。这就证明了流水作业调度问题具有最优子结构的性质。

2. 流水作业调度的递归结构

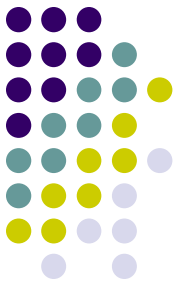


由流水作业调度问题的最优子结构性质可知

$$T(N, 0) = \min_{1 \leq i \leq n} \{a_i + T(N - \{i\}, b_i)\}$$

$$T(S, t) = \min_{i \in S} \{a_i + T(S - \{i\}, b_i + \max\{t - a_i, 0\})\}$$

式中， $\max\{t - a_i, 0\} = \max\{t, a_i\} - a_i$ 表示，
作业*i*必须在 $\max\{t, a_i\}$ 时间之后才能在机
器M2上开工。



3. Johnson不等式

对递归式的分析表明，算法可进一步得到简化。

设 π 是作业集 S 在机器 M_2 的等待时间为 t 时的任一最优调度。若 $\pi(1)=i, \pi(2)=j$ 。则由动态规划递归式可得：

$$T(S,t)=a_i+T(S-\{i\},b_i+\max\{t-a_i,0\})=a_i+a_j+T(S-\{i,j\},t_{ij})$$

其中，

$$\begin{aligned}t_{ij} &= b_j + \max\{b_i + \max\{t - a_i, 0\} - a_j, 0\} \\&= b_j + b_i - a_j + \max\{\max\{t - a_i, 0\}, a_j - b_i\} \\&= b_j + b_i - a_j + \max\{t - a_i, a_j - b_i, 0\} \\&= b_j + b_i - a_j - a_i + \max\{t, a_i + a_j - b_i, a_i\}\end{aligned}$$

如果作业 i 和 j 满足 $\min\{b_i, a_j\} \geq \min\{b_j, a_i\}$ ，则称作业 i 和 j 满足Johnson不等式。

3. Johnson不等式



交换作业*i*和作业*j*的加工顺序，得到作业集*S*的另一调度，它所需的加工时间为 $T'(S, t) = a_i + a_j + T(S - \{i, j\}, t_{ji})$

其中， $t_{ji} = b_j + b_i - a_j - a_i + \max\{t, a_i + a_j - b_j, a_j\}$

当作业*i*和*j*满足Johnson不等式时，有

$$\max\{-b_i, -a_j\} \leq \max\{-b_j, -a_i\}$$

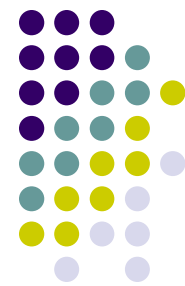
$$\rightarrow a_i + a_j + \max\{-b_i, -a_j\} \leq a_i + a_j + \max\{-b_j, -a_i\}$$

$$\rightarrow \max\{a_i + a_j - b_i, a_i\} \leq \max\{a_i + a_j - b_j, a_j\}$$

$$\rightarrow \max\{t, a_i + a_j - b_i, a_i\} \leq \max\{t, a_i + a_j - b_j, a_j\}$$

$$\rightarrow t_{ij} \leq t_{ji} \rightarrow T(S, t) \leq T'(S, t)$$

3. Johnson不等式



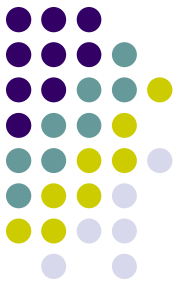
由此可见，当作业*i*和作业*j*不满足**Johnson**不等式时，交换它们的加工顺序后，不增加加工时间。

对于流水作业调度问题，必存在最优调度 π ，使得作业 $\pi(i)$ 和 $\pi(i+1)$ 满足**Johnson**不等式。

进一步还可以证明，调度满足**Johnson**法则当且仅当对任意*i*<*j*有

$$\min\{b_{\pi(i)}, a_{\pi(j)}\} \geq \min\{b_{\pi(j)}, a_{\pi(i)}\}$$

由此可知，**所有满足Johnson法则的调度均为最优调度。**



4. 算法描述

流水作业调度问题的Johnson算法

- (1) 令 $N_1 = \{i \mid a_i < b_i\}$, $N_2 = \{i \mid a_i \geq b_i\}$;
- (2) 将 N_1 中作业依 a_i 的非减序排序;
将 N_2 中作业依 b_i 的非增序排序;
- (3) N_1 中作业接 N_2 中作业构成满足Johnson法则的最优调度。

算法复杂度分析：算法的主要计算时间花在对作业集的排序。因此，在最坏情况下算法所需的计算时间为 $O(n \log n)$ ，所需的空间为 $O(n)$ 。

本周任务



1. 习题3-14
2. 对维数为序列5, 10, 3, 12, 5, 50, 6的各矩阵，找出其矩阵链乘积的一个最优加全部括号。
3. 说明如何在不用表b的情况下，通过已计算出的表c和原始序列 $X_m = \langle x_1, x_2, \dots, x_m \rangle$ 和 $Y_n = \langle y_1, y_2, \dots, y_n \rangle$ ，在 $O(m+n)$ 时间内重构一个LCS.
4. 使用替换法证明P63递归公式的解为 $\Omega(2^n)$

$$P(n) = \begin{cases} 1 & \text{if } n = 1 \\ \sum_{k=1}^{n-1} P(k)P(n-k) & \text{if } n \geq 2 \end{cases}$$